

Folio—A Personal Finance Platform

ISA 345 Sp. 2026

Andrew Joseph, Graham Boswell, Jacob Farmer, Peter Franz

Project Overview

Folio is a fintech subscription service where users pay a monthly or annual membership fee to access tiered personal finance tools including budgeting dashboards, savings goal tracking, credit score monitoring, and basic investment and brokerage tools. Our target customer is someone who wants a single dashboard that consolidates everyday budgeting with light-touch investing, rather than juggling separate tools for each.

Services are provided for subscribers in two tiers—basic and premium.

A basic subscription costs \$4 per month. It includes the following core features:

- Up to two linked financial or banking accounts.
- Access to the budgeting dashboard with spending categories.
- Monthly spending reports and savings summaries.
- Spending management support for personal finance goals.

A premium subscription costs \$12 per month. It includes all of the basic features as well as the follow tools:

- Unlimited linked financial or banking accounts.
- AI-powered spending insights and recommendations.
- Basic investment & brokerage capabilities.
- Advising and financial planning from a Folio certified wealth advisor.

Suppose a new subscriber purchased the premium subscription to receive both budgeting and brokerage services. They want to link a checking account from their local bank to the Folio app to receive AI spending insights and use the budgeting dashboard. The subscriber also wants to open a brokerage account with a certified wealth advisor.

Folio's day-to-day operation is organized around four interlocking activities. First, subscriber onboarding covers the registration of a new user, the selection of a basic or premium plan, and the linking of one or more external bank or brokerage accounts so that transactions can be aggregated. When a new user registers, the system creates a new record in the Subscriber table and associates them with a SubscriptionLevel. If they are premium, the system also triggers the creation of a corresponding Cash account record.

Second, budgeting and personal finance management covers the categorization of imported transactions, the creation and tracking of savings goals, and the generation of monthly spending and savings reports. When a user links a bank account, the system inserts a record into the BankBalance table and creates a relationship in the SubscriberBank associative entity to link that balance to the specific SubscriberID.

Third, investing and brokerage covers the funding of a Folio brokerage account, the placement of buy and sell orders for supported securities. When a user buys a stock, the system deducts the StockPrice from the CashAmount in the Cash table and inserts a new record into the CashStock table to track the new ownership.

Fourth, advisory services cover the matching of premium subscribers to certified wealth advisors, the scheduling of advising sessions, and the billing of advisor fees against the subscriber's brokerage cash balance.

Database Design and Implementation

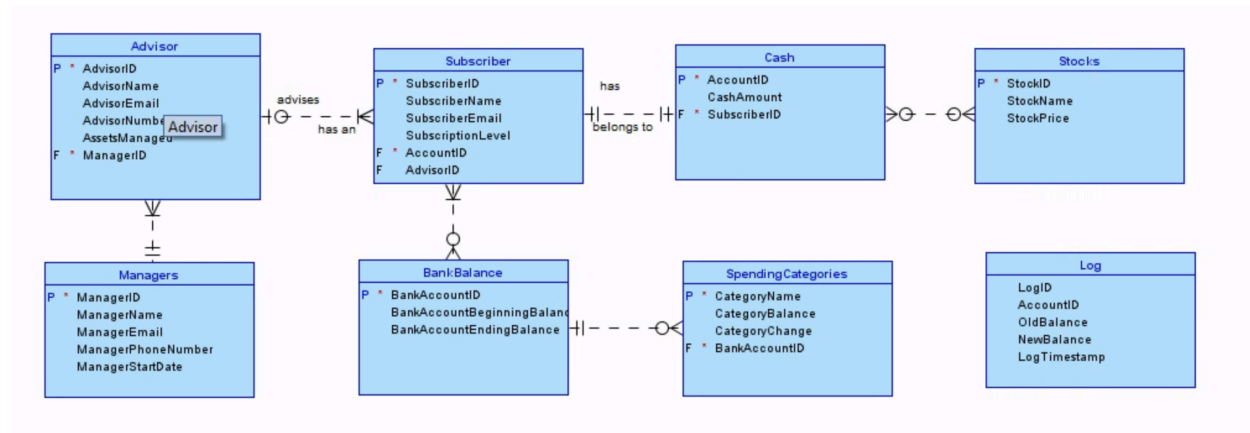
The key business rules that we had in mind during the construction of the database are as follows:

Advisor - Subscriber relationship: Subscribers can only have an advisor if they are a premium subscriber. Each advisor can have many subscribers they advise and each subscriber can have no more than one advisor.

Subscriber - BankBalance relationship: Each Basic Subscriber can connect up to two bank accounts, so it needs to be a many to many relationship. Premium Subscribers can connect as many bank accounts as they want.

Subscriber - Cash relationship: Only premium subscribers can have a cash account, so the relationship needs to be optional. Each subscriber can also only have one cash account, so it is a one to one relationship.

Cash - Stocks relationship: Only premium subscribers can buy stocks with the cash in their cash account. Each subscriber can buy multiple stocks, and each stock can be bought by multiple subscribers. This is a nonmandatory many to many relationship.



The initial logical model for our database was designed to support Folio, a personal investing application, by capturing user relationships and financial account data, both exogenous and endogenous to the financial service. At the core of this model is the Subscriber entity, which stores key user information such as name, email, and subscription level. Subscribers are able to link multiple third-party bank accounts to the platform, with associated financial data stored in the BankBalance entity.

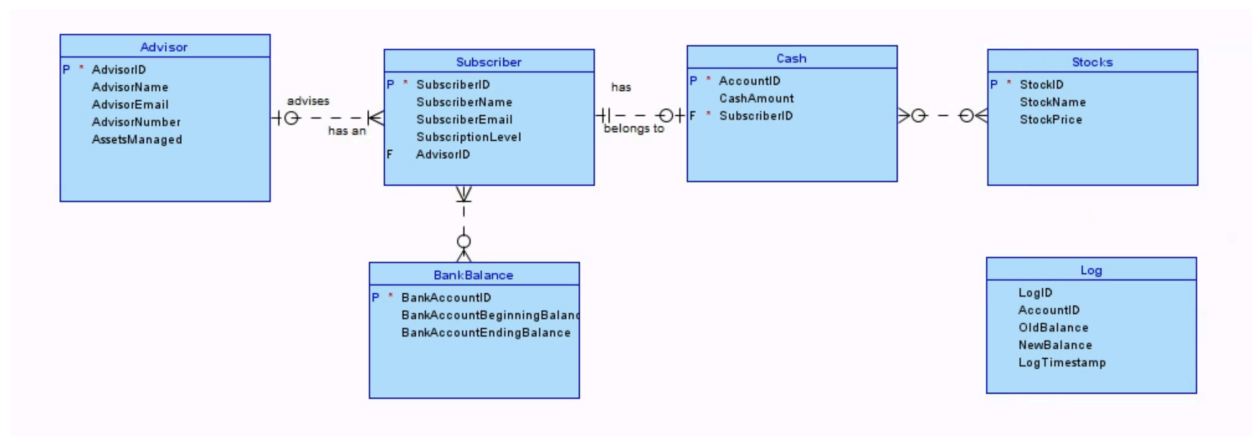
To provide users with insights into their financial activity over time, the BankBalance entity is connected to a SpendingCategory entity. This relationship enables tracking of expenditures across various categories, allowing subscribers to analyze month-over-month spending trends. The inclusion of beginning and ending balances within the BankBalance table further supports this functionality, illustrating changes in (third-party) account balances over time.

For premium subscribers, the model includes additional features to support investment and advisory services. The Advisor entity stores information about financial advisors, including their name, contact details, assets under management, and reporting structure. Each advisor can manage multiple subscribers, while each subscriber is assigned to a single advisor. A separate Manager entity is introduced to represent the hierarchical relationships among advisors within the organization.

Premium functionality also included financial account tracking through the Cash and Stocks entities. The Cash entity maintained records of funds held within a subscriber's account on the platform. The Stocks entity captured investment data, including stock names and market prices, reflecting holdings owned by subscribers. This relationship was many-to-many, as each subscriber could own multiple stocks, and each stock could be owned by multiple subscribers.

Additionally, the model included a Log table, which was not directly connected to other entities. The purpose of this table was to track transactional activity within the system. It stored attributes

such as LogID, AccountID, the previous balance, the updated balance, and a timestamp indicating when the transaction occurred. This structure supports auditing and historical tracking of account changes over time.



In the final iteration of our logical model, we streamlined the database design to improve efficiency, maintainability, and alignment with business requirements. Specifically, the Manager and SpendingCategory entities were removed because they introduced additional complexity without providing sufficient functional value to the application.

During the normalization process, we also identified several important functional dependencies. In both the Advisor and Subscriber tables, we observed the dependency:

- Email → PhoneNumber

Because each email address is unique to a single user, the corresponding phone number is also uniquely determined by that email. Although this represents a functional dependency, it does not create a normalization issue in this context because both attributes are intended to describe the same entity and maintain a one-to-one relationship.

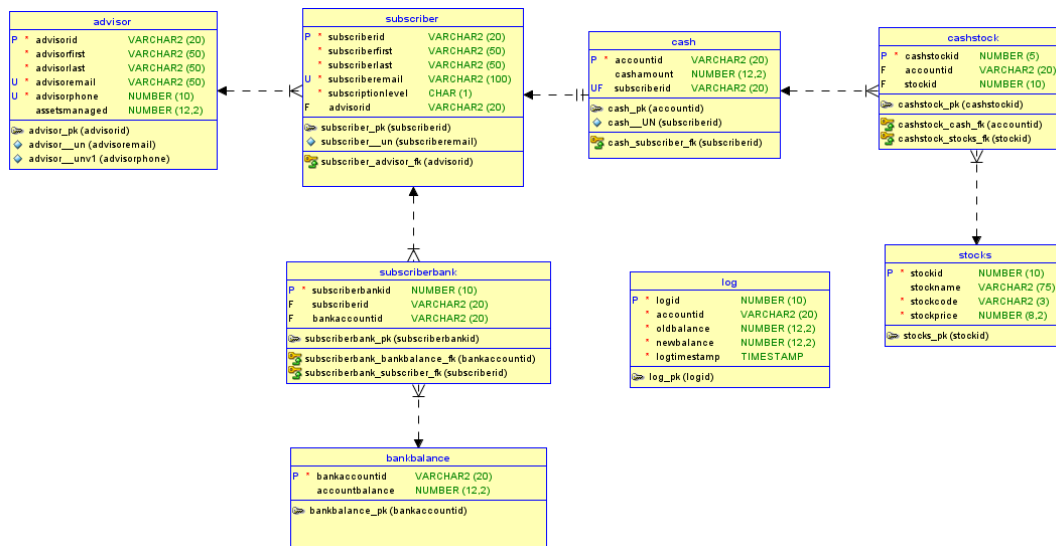
Additionally, within the BankBalance structure, we identified redundancy involving balance-related attributes. The original design stored both BeginningBankBalance and CurrentBankBalance in both the primary table and the associated log table, creating dependencies such as:

- BankBalanceRecordID → BeginningBankBalance, CurrentBankBalance

This redundancy introduced the risk of update anomalies, since any modification to the current balance would also require corresponding updates in the log to maintain consistency. To eliminate this issue, we revised the design to store only the CurrentBankBalance in the primary

table. Historical and derived balance information can instead be reconstructed through database views or queries against the log data when needed.

Overall, these refinements produced a more focused, maintainable, and normalized data model while preserving the core functionality required by the application.



The relational model was developed directly from the finalized logical model, with modifications to ensure proper implementation within a relational database structure. One of the primary differences between the logical and relational models is the introduction of associative (junction) tables to resolve many-to-many relationships.

Specifically, the SubscriberBank table was created to establish the relationship between Subscriber and BankBalance, allowing each subscriber to be linked to multiple bank accounts and vice versa. Similarly, the CashStock table was introduced to connect the Cash and Stocks entities, enabling accurate representation of investment holdings across subscriber accounts.

Additional refinements were made to enhance data tracking and integrity. The Stocks entity was updated to include a StockCode attribute, providing a unique identifier for each stock. Furthermore, the BankBalance entity was extended with an AccountRollingBalance attribute, allowing for more granular tracking of account balances on days beyond just the beginning and end of each month.

These enhancements ensure that the relational model is both normalized and capable of supporting the functional requirements of the application while maintaining data consistency and scalability.

For a full breakdown of the entities and their associated attributes, please visit the database dictionary in the attached appendix.

Operational SQL

Included below are five useful SQL queries that give insight into the data held within the database. Each query demonstrates how users of the data could benefit from analysis with this database.

Our first query highlights heavier users of the Folio app—those that maintain higher balances in the cash accounts. This code returns the identifying information for all users that have cash account balances greater than the average cash account balance for all users. Currently, four users are returned, and we identify that all of them are premium subscribers.

```
SELECT SUBSCRIBER.SubscriberID, SUBSCRIBER.SubscriberFirst,  
SUBSCRIBER.SubscriberLast, SUBSCRIBER.SubscriptionLevel, CASH.CashAmount,  
Advisor.AdvisorFirst, Advisor.AdvisorLast  
FROM SUBSCRIBER  
JOIN CASH ON SUBSCRIBER.SubscriberID = CASH.SubscriberID  
JOIN ADVISOR ON SUBSCRIBER.AdvisorID = ADVISOR.AdvisorID  
WHERE CASH.CashAmount > (SELECT AVG(CashAmount) FROM Cash)  
ORDER BY CASH.CashAmount DESC;
```

Our second query shows user accounts where the cash account balances have changed since sign-up (either from deposits or withdrawals). This query returns every cash balance change recorded in the audit log, showing which subscribers transacted, what direction the money moved, and the size of each change. Withdrawals are set to appear first. This query produces an audit trail of all cash account activity which could support compliance, fraud prevention, and customer support.

```
SELECT SUBSCRIBER.SubscriberID, SUBSCRIBER.SubscriberFirst,  
SUBSCRIBER.SubscriberLast, LOG.OldBalance, LOG.NewBalance, (LOG.NewBalance -  
LOG.OldBalance) AS "Net Change"  
FROM SUBSCRIBER  
JOIN CASH ON SUBSCRIBER.SubscriberID = CASH.SubscriberID  
JOIN LOG ON CASH.AccountID = LOG.AccountID
```

```
WHERE LOG.OldBalance != LOG.NewBalance
ORDER BY (LOG.NewBalance - LOG.OldBalance), LOG.NewBalance;
```

The third query shows users that have not linked bank accounts to the Folio app. Every user should connect at least one bank account to the Folio app to handle cash transfers and deposits. This query returns all users where the bank account id is null, allowing the managers to identify incomplete profiles or the profiles of users who are not fully utilizing the Folio resources.

```
SELECT SUBSCRIBER.SubscriberID, SUBSCRIBER.SubscriberFirst,
SUBSCRIBER.SubscriberLast, SUBSCRIBER.SubscriberEmail,
SUBSCRIBER.SubscriptionLevel
FROM SUBSCRIBER
LEFT OUTER JOIN SUBSCRIBERBANK ON SUBSCRIBER.SubscriberID =
SUBSCRIBERBANK.SubscriberID;
```

The fourth query identifies the amount of subscribers for each subscription level, as well as the average balance for each subscription level and the highest and lowest balances for those subscription levels. This query allows the data manager to view key statistics for analyzing the success of the two subscription levels.

```
SELECT SUBSCRIBER.SubscriptionLevel,
COUNT(SUBSCRIBER.SubscriberID) AS "Total Subscribers",
SUM(CASH.CashAmount) AS "Total Cash",
ROUND(AVG(CASH.CashAmount), 2) AS "Average Cash",
MAX(CASH.CashAmount) AS "Highest Balance",
MIN(CASH.CashAmount) AS "Lowest Balance"
FROM SUBSCRIBER
INNER JOIN CASH ON SUBSCRIBER.SubscriberID = CASH.SubscriberID
GROUP BY SUBSCRIBER.SubscriptionLevel;
```

With our final query, we wanted to investigate which advisors are higher performing, meaning which advisors oversee clients with larger cash balances. Specifically, we wanted to find financial advisors whose average client cash balance exceeds \$10,000. You can view the query below.

```
SELECT Advisor.AdvisorID, AdvisorFirst, AdvisorLast, AVG(Cash.CashAmount)
FROM ADVISOR JOIN SUBSCRIBER ON Advisor.AdvisorID = Subscriber.AdvisorID
JOIN CASH ON Cash.SubscriberID = Subscriber.SubscriberID
GROUP BY Advisor.AdvisorID, AdvisorFirst, AdvisorLast
HAVING AVG(Cash.CashAmount) > 10000
ORDER BY AVG(Cash.CashAmount) DESC;
```


Additionally, we created two INDEXES, a VIEW and a TRIGGER to support the use of this data. Our first INDEX provides the ability to track changes in user balances over time in the LOG entity, leaving an audit trail for database users to follow.

```
CREATE INDEX AccountTransactionHistory  
ON LOG(AccountID, LogTimestamp);
```

Our second INDEX is meant to support queries that deal with the cash table. This INDEX is used when a customer or advisor checks the balance of a cash account, or in the development of holding reports analyzing client information. This INDEX would be especially useful for aggregate operations within these reports—looking at calculations or comparison of balances across accounts, subscription tiers, etc.

```
CREATE INDEX SubscriberCashBalances  
ON CASH(SubscriberID, CashAmount);
```

The true financial picture of Folio subscribers is spread across multiple entities within our database. This VIEW joins each subscriber's cash account balance with each of their linked bank account balances. This gives advisors a breakdown of where a client's assets sit, useful for reporting, onboarding decisions, and investment recommendations.

```
CREATE VIEW SubscriberTotalAssets AS  
SELECT Subscriber.SubscriberID, Subscriber.SubscriberFirst, Subscriber.SubscriberLast,  
Cash.CashAmount, BankBalance.AccountBalance, (Cash.CashAmount +  
BankBalance.AccountBalance) AS "Total Assets"  
FROM SUBSCRIBER  
JOIN CASH ON Subscriber.SubscriberID = Cash.SubscriberID  
JOIN SUBSCRIBERBANK ON Subscriber.SubscriberID = SubscriberBank.SubscriberID  
JOIN BANKBALANCE ON SubscriberBank.BankAccountID = BankBalance.BankAccountID;
```

By creating the LogBalanceChange TRIGGER, a Folio user's log balance changes automatically when their cash balance is updated following an investment or other transaction. Without this, the log entity would need to be populated manually, risking incomplete or missing audit records. This would be helpful in cases of regulatory compliance checks or resolving customer disputes.

```
CREATE TRIGGER LogBalanceChange  
AFTER UPDATE OF CashAmount ON CASH  
FOR EACH ROW  
BEGIN  
    INSERT INTO LOG (LogID, AccountID, OldBalance, NewBalance, LogTimestamp)  
    VALUES (
```

```
(SELECT MAX(LogID) + 1 FROM LOG),  
:OLD.AccountID,  
:OLD.CashAmount,  
:NEW.CashAmount,  
SYSTIMESTAMP  
);  
END;  
/
```

All operation SQL statements can also be found in the attached .sql file.

Deployment

We have created a SQL agent to interact with the created database using natural language. The SQL agent will benefit Folio in many ways, including reducing mistakes on the backend from querying data, allow for faster decision making from more accessible data, and provide greater consistency in reporting. Proper authentication will also allow Folio to have a clear audit trail of who used the agent to access sensitive Folio data and prevent unauthorized access. Ultimately, this SQL agent will turn Folio's databases into an organizational resource for better data management. You can access the SQL agent at the following link:

<https://miamiuniversity.app.n8n.cloud/webhook/fbf7fc46-464f-4079-8314-d1db87b882a1/chat>

We also created a web application using Python, as a model prototype for an app all Folio users could download to manage their relationship with Folio. This application includes a personalized dashboard, connected to all relevant accounts, that displays the bank balance of all connected bank accounts, current Folio cash balance, and the portfolio value of all investments made through Folio. Additionally, a user can transfer funds from connected bank accounts to their Folio cash account, and prompt investments in shares of stocks within the database.

The account tab displays all separate bank accounts currently linked to Folio for a user, alongside individual account balances for each. The investing tab lists all past investment history for a user, and a prototype of a visualization of this information. If a Folio user has a premium subscription, their advisor information is listed in the Advisor tab. The My Folio Account tab displays a user's current subscription model, and allows the user to "log-out". For prototype purposes, "logging out" means typing in a different subscriber ID, in order to view a different subscriber's information. The security concerns raised by this would be taken away within a commercialized, published application.

You can access a prototype of the fully function web application here:

<https://ai.studio/apps/430a70e7-0d8d-41a8-9fce-4093986b1154>

Conclusions & Lessons Learned

Folio is a comprehensive personal finance application designed to help users efficiently track and manage their bank accounts over time. Through our database design, we successfully achieved the goal of creating a scalable and reliable platform that supports everyday financial management as well as long-term investment growth. By centralizing account balances, transaction history, and portfolio information into a structured and normalized database, Folio would enable users to make more informed financial decisions and monitor their financial health with greater accuracy and convenience.

In addition to improving organization and accessibility, the database was designed to enhance productivity by reducing redundancy, maintaining data consistency, and supporting efficient data retrieval. These optimizations allow the application to quickly provide meaningful financial insights and support future expansion of features. Furthermore, Folio's premium subscription services give users an easy and streamlined way to invest and grow their portfolios, making the platform not only a budgeting and tracking tool, but also a resource for long-term financial planning and decision-making. Overall, the final database design provides a maintainable, efficient, and user-focused foundation for a successful personal finance application.

Appendix

Database Dictionary:

Advisor:

- AdvisorID: A unique numeric key which identifies each advisor
- AdvisorFirst: The first name of the advisor
- AdvisorLast: The last name of the advisor
- AdvisorEmail: The email address of the advisor
- AdvisorPhone: The phone number of the advisor
- AssetsManaged: A numeric value which represents the book value of the assets managed by that specific advisor

Subscriber:

- SubscriberID: A unique numeric key which identifies each subscriber
- SubscriberFirst: The first name of the subscriber
- SubscriberLast: The last name of the subscriber
- SubscriberEmail: The email address of the subscriber
- SubscriptionLevel: A code that identifies whether the user has a basic or premium
- AdvisorID: A foreign key that links the subscriber to their assigned financial advisor.

Cash:

- AccountID: A unique numeric key which identifies the internal cash account.
- CashAmount: The total amount of liquid funds currently held in the account.
- SubscriberID: A unique foreign key that identifies the owner of the cash account.

Stocks:

- StockID: A unique numeric key used to identify a specific stock.
- StockName: The full name of the company or security.
- StockCode: A unique three character ticker symbol used to identify the stock.
- StockPrice: The current market price for a single share of the stock.

CashStock:

- CashStockID: A unique key identifying a specific record of stock ownership in an account.
- AccountID: A foreign key linking the record to a specific cash account.
- StockID: A foreign key linking the record to a specific stock.

SubscriberBank:

- SubscriberBankID: A unique key identifying the link between a subscriber and an external bank account.
- SubscriberID: A foreign key identifying which subscriber owns the bank account.
- BankAccountID: A foreign key identifying the specific external account being linked.

BankBalance:

- BankAccountID: A unique key identifying a specific external bank account.
- AccountBalance: The current total balance held in the linked external account.

Log:

- LogID: A unique numeric key used to identify each entry in the audit trail.
- AccountID: The identifier for the specific account where a balance change occurred.
- OldBalance: The balance of the account recorded before the transaction was processed.
- NewBalance: The balance of the account recorded after the transaction was completed.
- LogTimestamp: The specific date and time the transaction was recorded in the system.